NATIONAL UNIVERSITY
OF COMPUTER AND EMERGING SCIENCES
CHINIOT - FAISALABAD CAMPUS



CS-1002
Programming Fundamentals

Semester Fall 2026

FAST School of Computing

Submission Instructions

- Solve every question in C++, using a **separate .cpp file for each question** (e.g., Q1.cpp, Q2.cpp, …, Q10.cpp).
- Create a word file containing the C++ code and the screenshot of the output for each question.
- The use of **any kind of loop** (for, while, do-while) is **NOT allowed** in any question.
- Put all ten .cpp files in one folder, submit them as a single **.zip file** and a separate word file. So **Final Submission (.zip file + word file).**
- This is an individual assignment; collaboration is not allowed.
- Submit your own original work. **Plagiarism will not be tolerated**, either done from the internet or by any fellow.
- Any submission that does not adhere to these instructions will not be evaluated.
- **Deadline: October 10, 2026**

Question 1: Gym Workout Plan

Write a C++ program that prints a gym's workout plan for a given day, based on a day number entered by the user (1 = Monday, 2 = Tuesday, ..., 7 = Sunday).
Requirements:

- 1 (Monday): "Chest and Triceps"
- 2 (Tuesday): "Back and Biceps"
- 3 (Wednesday): "Rest Day"
- 4 (Thursday): "Legs"
- 5 (Friday): "Shoulders and Abs"
- 6 (Saturday): "Cardio"
- 7 (Sunday): "Rest Day"
- Any other number: print "Invalid day number"

SAMPLE INPUT 1 day = 4 SAMPLE OUTPUT 1 Legs	SAMPLE INPUT 2 day = 7 SAMPLE OUTPUT 2 Rest Day

---

Question 2: FESCO Residential Electricity Bill Calculation

You are tasked with developing a program that calculates the monthly electricity bill for residential customers of FESCO (Federal Electric Supply Company) based on their monthly consumption in units. The calculation differs based on the number of units consumed, and it includes applicable taxes.

Requirements:

1. The user will input the following:
   - The number of units consumed in a month (should be a non-negative integer).
2. The bill calculation will follow these rules:
   - 0 to 100 units: PKR 5 per unit
   - 101 to 300 units: PKR 7 per unit
   - 301 to 400 units: PKR 24.40 per unit
   - 401 to 500 units: PKR 24.91 per unit
   - 501 to 600 units: PKR 26.15 per unit
   - 601 to 700 units: PKR 27.59 per unit
3. Taxes:
   - The total bill will have an additional 10% tax (for general tax).

4. The program should ensure the following:
- Input validation to ensure that the number of units consumed is a non-negative integer.

5. The program should calculate the total bill based on the above rates, apply the taxes, and display the calculated amount to the user.

Example Input and Output:

Input:
Enter the number of units consumed: 350

Output:
The total electricity bill is: PKR 3432

Explanation:
First 100 units: 100 * 5.0 = PKR 500
Next 200 units: 200 * 7.0 = PKR 1400
Next 50 units: 50 * 24.40 = PKR 1220
Total bill before tax: 500 + 1400 + 1220 = PKR 3120
10% tax: 3120 * 0.10 = PKR 312
Total bill after tax: 3120 + 312 = PKR 3432

Note: Make sure to handle the calculation logic using if-else statements to determine the applicable rates based on the number of units consumed and apply the tax calculations correctly.

Question 3: Fitness App Calorie Burn Estimator

Write a C++ program that estimates the total calories burned during a workout session, based on the workout intensity (a character: 'L', 'M', or 'H') and the session duration in minutes (an integer).

Requirements:

- If intensity is 'L' (Low): the burn rate is 4 calories per minute, regardless of duration.
- If intensity is 'M' (Medium): the burn rate is 6 calories per minute if duration is 30 minutes or less, and 7 calories per minute if duration is more than 30 minutes.

- If intensity is 'H' (High): the burn rate is 10 calories per minute if duration is 20 minutes or less; 12 calories per minute if duration is more than 20 but no more than 45 minutes; and 9 calories per minute if duration is more than 45 minutes (fatigue reduces efficiency on long high-intensity sessions).
- If intensity does not match 'L', 'M', or 'H', print **"Invalid intensity level"** and do not calculate anything further.
- For a valid intensity, output the total calories burned, calculated as **burn rate × duration**.

Notes:

- 'L' represents Low intensity.
- 'M' represents Medium intensity.
- 'H' represents High intensity.
- Checking whether duration falls within a range (e.g., "more than 20 but no more than 45") must be done using **nested if-else**, never with && or ||.

SAMPLE INPUT 1 intensity = 'H' duration = 30 SAMPLE OUTPUT 1 Calories burned: 360	SAMPLE INPUT 2 intensity = 'M' duration = 45 SAMPLE OUTPUT 2 Calories burned: 315

Question 4: Airline Baggage Fee Calculator

Write a C++ program that calculates the checked-baggage fee for an airline passenger, based on their ticket class (a single character: 'E' for Economy, 'P' for Premium Economy, 'B' for Business, 'F' for First Class) and the number of checked bags (an integer).

Requirements:

• Economy ('E'): the first bag is free; the 2nd bag costs $35; every bag beyond the 2nd costs $75 each.

• Premium Economy ('P'): the first 2 bags are free; every bag beyond that costs $60 each.

• Business ('B'): the first 3 bags are free; every bag beyond that costs $50 each.

• First Class ('F'): all bags are free, regardless of how many are checked.

• Any other character: print "Invalid ticket class" and do not calculate a fee.

Notes:

• For Economy with bags ≥ 2, the fee is $35 for the 2nd bag plus $75 for every bag after

that (e.g., 3 bags = $35 + $75 = $110).
• You do not need a loop — the fee for a given bag count can be computed directly with arithmetic and simple if-else inside each case.

SAMPLE INPUT 1 ticketClass = 'E' bags = 3 SAMPLE OUTPUT 1 Baggage fee: $110	SAMPLE INPUT 2 ticketClass = 'B' bags = 2 SAMPLE OUTPUT 2 Baggage fee: $0

Question 5: Recursive Rightmost Digit Manipulation

You are tasked with creating a program that takes a 6-digit number as input from the user and repeatedly modifies it based on the rightmost digit until only one digit remains. The rightmost digit is discarded after each step, and the process continues with the remaining number.

Process:

1. The user will input a 6-digit number.
2. The program checks the rightmost digit of the number:
   - If the last digit is even, add 1 to the remaining number (after discarding the last digit).
   - If the last digit is odd, subtract 1 from the remaining number (after discarding the last digit).
   - If adding 1 results in the remaining number gaining an extra digit, subtract 2 instead.
3. Discard the rightmost digit and apply the same process to the new modified number.
4. Repeat this process until only 1 digit remains.
5. The program will then print the final digit.

Example 1	**Example 2**
Input: 999992 • The rightmost digit is 2 (even), so you add 1 to the remaining number (99999), getting 100000. Since adding 1 causes a digit overflow, you subtract 2 instead, resulting in 99997. • Now, the number is 99997. The rightmost digit is 7 (odd), so you	**Input: 123456** • The rightmost digit is 6 (even), so you add 1 to the remaining number (12345), getting 12346. • Now, the number is 12346. The rightmost digit is 6 (even), so you add 1 to the remaining number (1234), getting 1235.

subtract 1 from the remaining number (9999), getting 9998. • Now, the number is 9998. The rightmost digit is 8 (even), so you add 1 to the remaining number (999), getting 1000. This adds a digit, so you subtract 2 instead, getting 997. • Now, the number is 997. The rightmost digit is 7 (odd), so you subtract 1 to the remaining number (99), getting 98. • Now, the number is 98. The rightmost digit is 8 (even), so you add 1 to the remaining number (9), getting a 10. Since adding 1 causes a digit overflow, you subtract 2 instead, resulting in 7. • The final result is 7.	• Now, the number is 1235. The rightmost digit is 5 (odd), so you subtract 1 from the remaining number (123), getting 122. • Now, the number is 122. The rightmost digit is 2 (even), so you add 1 to the remaining number (12), getting 13. • Now, the number is 13. The rightmost digit is 3 (odd), so you subtract 1 from the remaining number (1), getting 0. • The final result is 0.

Question 6: Chess Master 20706

You are contributing to a chess bot designed to challenge Magnus Carlsen. Because of limited resources, you may not use loops or arrays. Your task is to build the check engine, which determines whether the king is in check.

Input

King: position

Normal piece: position, color, and type

Positions use standard chess notation: file A-H, rank 1-8. Color is B (Black) or W (White). Piece type uses the standard abbreviations: Q, R, B, N, P. The king is always the opposite color of the normal piece.

Task

Validate the positions. They are invalid if either is off the board or both pieces occupy the same square.

If valid, determine whether the normal piece is giving check to the king, based on its position, type, and color.

Example Input

King: A 1
Normal Piece: A 7 , B , R

Output
The Positions are Valid!
The Black Rook is giving a check to the White King

Note: Standard chess notation is mandatory. Ranks must be a number from 1 to 8, and files a letter from A to H.

Question 7: MMA's LAYERS BAKESHOP!

MMA has appointed you the manager of his bakery to test your management skills. The bakery sells Muffins, Mango Sundaes, and Apple Tarts. Each day, five customers are already at the bakery. They hold some items and want to buy more. Customers are served in strict order.

Write a program that decides which customers can be served, then reports the day's results.

CONSTRAINT: Loops and arrays are not allowed.

INPUT

1. Stock: the available quantity of each of the three items.
2. For each customer, in order:
   - Name (char)
   - Quantity of each item they already have (int)
   - Quantity of each item they want in total (int)

The program must use exception handling and validation checks. Quantities must be valid non-negative numbers, and the amount wanted of an item cannot be less than the amount already held.

PROCESS

For each customer, calculate the need for each item:

Need = Wanted - Already Have

• If the need for every item is less than or equal to the stock, the customer is served and the needed amounts are deducted from the stock.

• If the need for even one item is more than the stock, the customer is not served and the stock remains unchanged.

OUTPUT

After all five customers are processed, display:

• The number of customers served

• The serving rate as a percentage
• The names of the customers served

SAMPLE RUN
Stock: Muffins 10, Sundaes 5, Tarts 4

Customer	Have(M/S/T)	Want(M/S/T)	Need(M/S/T)	Result
A	2/1/0	5/2/1	3/1/1	Served
S	0/0/1	8/1/3	8/1/2	Not Served
H	1/0/0	4/4/3	3/4/3	Served
Z	0/1/0	2/2/0	2/1/0	Not Served
B	1/0/0	3/0/0	2/0/0	Served

Output:
Customers Served: 3
Serving Rate: 60%
Served Customers: A, H, B

Question 8: Valorant: The Perfect Buy

You are playing a competitive FPS game where your team's economy determines what equipment you can purchase before each round.

Input

Credits: credits currently available to the player
LossStreak: number of consecutive rounds lost
wonLastRound: whether the player won the previous round
aliveLastRound: whether the player survived the previous round
teammatesBuying: whether most teammates are buying this round

Step 1: Determine the Round Economy

If the player won the previous round check their credits:
If credits are 800 or less, display:
  "LOW FUNDS — Consider saving."
Otherwise, display:
  "WINNING ECONOMY — Full purchase possible."
If the player lost the previous round, check the lossStreak:
If lossStreak is 1, display:
  "1 LOSS — Low loss bonus."
If lossStreak is 2, display:

"2 LOSSES — Medium loss bonus."
If lossStreak is 3 or more, display:
"MAX LOSS BONUS — Strong loss bonus available."

Step 2 — Determine Whether the Player Can Afford a Full Buy
A Full Buy requires at least 4,000 credits.
if the player has 4,000 or more credits, check whether their teammates are also buying.
If teammates are buying, display:
"FULL BUY — Buy your main equipment."
Otherwise, display:
"TEAM NOT BUYING — Consider saving for the next round."
If the player has less than 4,000 credits, continue to Step 3.

Step 3: Determine Whether a Force Buy Is Possible
If the player cannot afford a Full Buy:
If the player has at least 2,500 credits, check the loss streak.
If lossStreak is 2 or more, display:
"FORCE BUY AVAILABLE — You have enough credits after consecutive losses."
Otherwise, display:
"PARTIAL BUY — Purchase only essential equipment."
If the player has less than 2,500 credits, continue to Step 4.

Step 4: Determine the Economy Buy
If the player has less than 2,500 credits:
If lossStreak is 3 or more, display:
"ECO ROUND — Save credits and use minimal equipment."
Otherwise:
If the player has at least 1,000 credits, display:
"LIGHT BUY — Purchase inexpensive equipment only."
Otherwise, display:
"HARD ECO — Save all available credits."

Step 5: Check the Player's Previous-Round Survival
After determining the purchase decision, check whether the player survived the previous round.
If the player survived, display:
"You saved some equipment from the previous round."
If the player did not survive, check whether the player is on a loss streak:

If loss streak is 2 or more, display:
"Lost equipment + loss bonus — Economy is recovering."
Otherwise, display:
"Lost equipment — Economy remains weak."

Step 6: Final Round Status

Finally, determine the overall economy status:
If the player has 4,000 or more credits AND their teammates are buying:
"ROUND STATUS: FULL BUY ROUND."
Otherwise, if the player has 2,500 or more credits AND has a loss streak of 2 or more:
"ROUND STATUS: FORCE BUY / RECOVERY ROUND."
Otherwise, if the loss streak is 3 or more:
"ROUND STATUS: ECONOMY ROUND — SAVE FOR FUTURE ROUNDS."
Otherwise:
"ROUND STATUS: SAVE / PARTIAL BUY."

Example 1	**Example 2**
credits = 4500 lossStreak = 0 wonLastRound = true aliveLastRound = true teammatesBuying = true WINNING ECONOMY — Full purchase possible. FULL BUY — Buy your main equipment. You saved some equipment from the previous round. ROUND STATUS: FULL BUY ROUND.	credits = 2000 lossStreak = 3 wonLastRound = false aliveLastRound = false teammatesBuying = true MAX LOSS BONUS — Strong loss bonus available. ECO ROUND — Save credits and use minimal equipment. Lost equipment + loss bonus — Economy is recovering. ROUND STATUS: ECONOMY ROUND — SAVE FOR FUTURE ROUNDS.

Question 9: BATTLESHIPS 101

You are tasked with creating a program that simulates a simple battleship game on a hidden 3×3 sea (rows 1 to 3, columns 1 to 3). The user chooses a map, then fires up to 4 shots by entering a row and a column. After every shot the program prints the result,

and at the end it prints the score and rank. **The program must not use loops, functions, arrays, or print any grid.**

Process:

1. The user inputs a map number (1 to 3). Use a switch to set the ship positions:

Map	Submarine (1 cell)	Destroyer (2 cells)
1	(1,1)	(2,3) and (3,3)
2	(3,1)	(1,2) and (1,3)
3	(2,2)	(3,1) and (3,2)

Any other map number: print **"Invalid map!"** and end the program.

2. Create three bool variables, subHit, dest1Hit and dest2Hit, all starting as false. Also create counters for shotsUsed, shipsSunk and score.

3. For each of the 4 shots (write the block once and copy it three times), the user inputs a row and a column. Add 1 to shotsUsed, then check the shot in this order:
   - If the row or column is below 1 or above 3, print "Out of range, shot wasted!"
   - If it matches a ship cell that is already hit, print "Already hit, shot wasted!"
   - If it matches the submarine, print "HIT! Submarine sunk!"
   - If it matches a destroyer cell, print "HIT!" If both destroyer cells are now hit, also print "Destroyer sunk!"
   - Otherwise, print "Miss!" followed by a radar hint (see step 4).

4. **Radar hint (on a miss only):** If the submarine is not sunk yet, find its distance from your shot:

   distance = |shotRow - subRow| + |shotCol - subCol|
   - Distance 1: Burning hot!
   - Distance 2: Warm
   - Distance 3 or more: Cold

If the submarine is already sunk, print no hint. (You may use abs() from <cstdlib>.)

5. Once both ships are sunk, the remaining shots are skipped and the program prints "Victory!"

6. **Scoring:**
   - Each ship cell hit: +10
   - Submarine sunk: +20 bonus
   - Destroyer sunk: +30 bonus

7. Print the shots used, ships sunk, score, and rank:

Condition	Rank
Both ships sunk in 3 shots or fewer	Admiral
Both ships sunk in 4 shots	Captain

Exactly one ship sunk	Sailor
No ship sunk	Cadet

Rules: Use switch for the map, at least one nested if, and at least five uses of && or ||. Invalid input must print a message and never crash.

Example 1	**Example 2**
Enter map (1-3): 1 Shot 1 - Enter row and column: 2 2 Miss! Warm Shot 2 - Enter row and column: 2 3 HIT! Shot 3 - Enter row and column: 3 3 HIT! Destroyer sunk! Shot 4 - Enter row and column: 1 1 HIT! Submarine sunk! Victory! Shots used: 4 Ships sunk: 2 Score: 80 Rank: Captain	Enter map (1-3): 2 Shot 1 - Enter row and column: 1 2 HIT! Shot 2 - Enter row and column: 1 2 Already hit, shot wasted! Shot 3 - Enter row and column: 1 3 HIT! Destroyer sunk! Shot 4 - Enter row and column: 2 2 Miss! Warm Shots used: 4 Ships sunk: 1 Score: 50 Rank: Sailor

---

Question 10: Welcome to NU-FAST!!!

You are freshly enrolled in FAST NU CFD and is tasked with creating a program that simulates a tiny 8-bit CPU. The user enters two 8-bit registers, AL and BL, and chooses one instruction. The program runs on 8 bits (results wrap around after 255), then prints the result and the CPU flags. **No loops, functions, or arrays are allowed.**

Process:

1. The user inputs AL and BL. Both must be between 0 and 255. Otherwise print **"Invalid input!"** and end the program.
2. The user inputs an instruction number. Use a switch to run it:

Choice	**Instruction**	**Operation**
1	ADD	AL + BL
2	SUB	AL - BL
3	AND	AL & BL
4	SHL	shift AL left by 1 (BL is ignored)

Any other number: print “**Invalid instruction**!” and end the program.
3. The result must fit in 8 bits. If it goes above 255 or below 0, wrap it around (300 becomes 44, and -5 becomes 251) (use 256 – 5 = 251).

4. Set three flags (each 0 or 1):
- **ZF (Zero):** 1 if the result is 0.
- **SF (Sign):** 1 if the result is 128 or more (the leftmost bit is 1).
- **CF (Carry):**
  ADD: 1 if the real sum is above 255.
  SUB: 1 if AL is smaller than BL.
  SHL: 1 if AL is 128 or more.
  AND: always 0.

5. Print the result and the three flags.

Rules: Use switch for the instruction, use at least one bitwise operator (&, <<), and use at least one nested if. Invalid input must print a message and never crash.

Example Input and Output:

```
Enter AL (0-255): 200
Enter BL (0-255): 100
Enter instruction (1-ADD 2-SUB 3-AND 4-SHL): 1
Result: 44
ZF=0 SF=0 CF=1
```

Best of Luck 😊